Lesson 08 Demo 06

Generating Workspaces in Terraform

Objective: To effectively utilize Terraform workspaces for environment separation, create, and manage a development workspace, deploy infrastructure within it, and switch between different workspaces.

Tools required: VS Code and Linux terminal

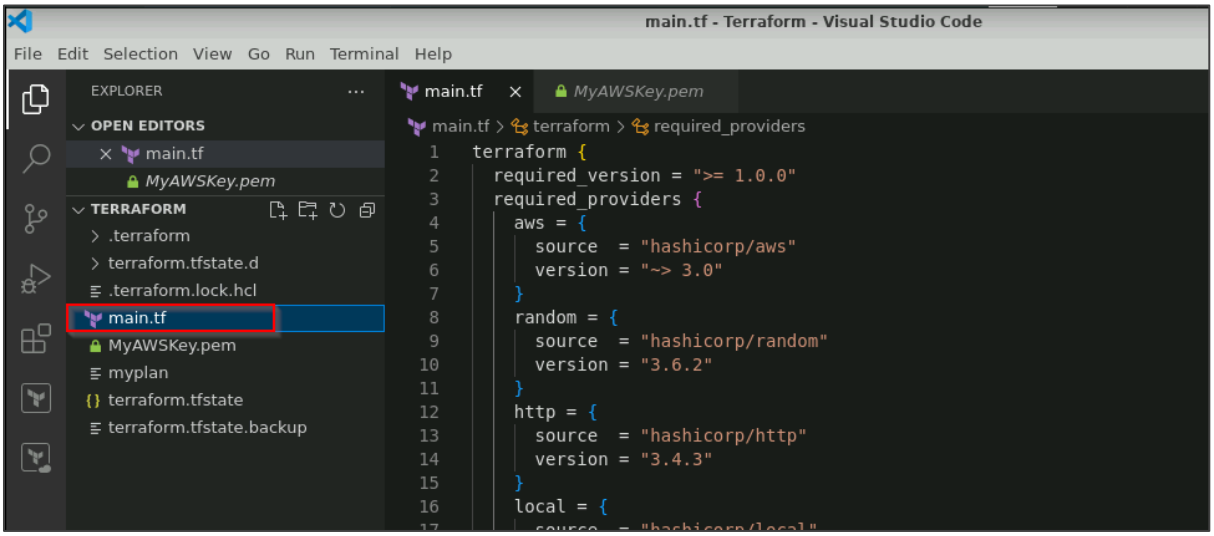
Prerequisites: Refer to demo 06

Steps to be followed:

1. Update the main.tf file
2. Check the Terraform workspaces
3. Generate a new Terraform workspace for the development state
4. Modify main.tf file
5. Navigate between workspaces

Step 1: Update the main.tf file

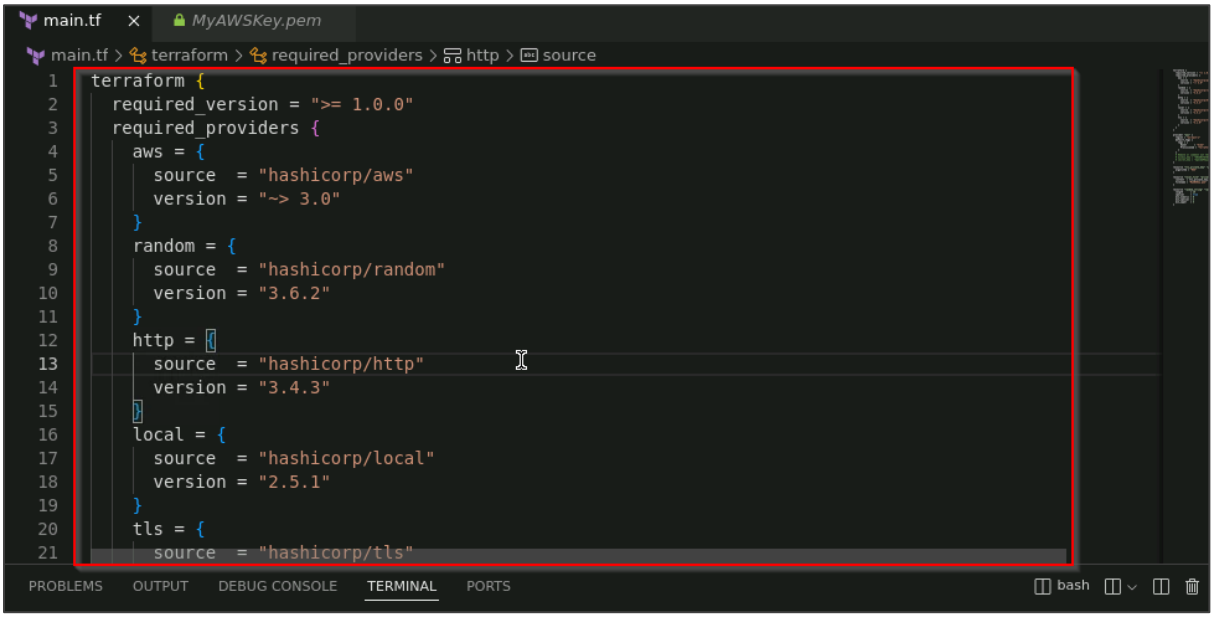
1.1 Open the **main.tf** file



1.2 Replace the **main.tf** contents with the code provided below for setting up and configuring your Terraform project to manage infrastructure resources on AWS:

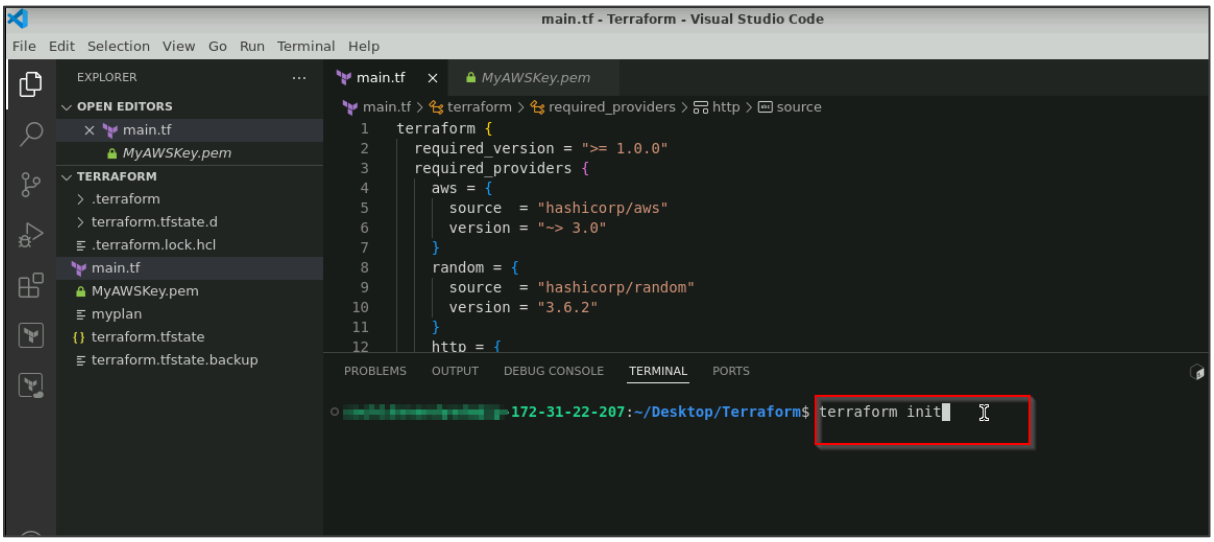
```
terraform {
  required_version = ">= 1.0.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 3.0"
    }
    random = {
      source  = "hashicorp/random"
      version = "3.6.2"
    }
    http = {
      source  = "hashicorp/http"
      version = "3.4.3"
    }
    local = {
      source  = "hashicorp/local"
      version = "2.5.1"
    }
    tls = {
      source  = "hashicorp/tls"
      version = "3.1.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
  default_tags {
    tags = {
      Owner      = "Acme"
      Provisioned = "Terraform"
    }
  }
```

```
  # access_key = "AKIARJTG7GGYJOVDRU3B"
  # secret_key = "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"
}

resource "tls_private_key" "generated" {
  algorithm = "RSA"
}

resource "local_file" "private_key_pem" {
  content  = tls_private_key.generated.private_key_pem
  filename = "MyAWSKey.pem"
}

resource "random_string" "random" {
  length      = 15
  special     = true
  min_numeric = 6
  min_special = 2
  min_upper   = 3
}
```

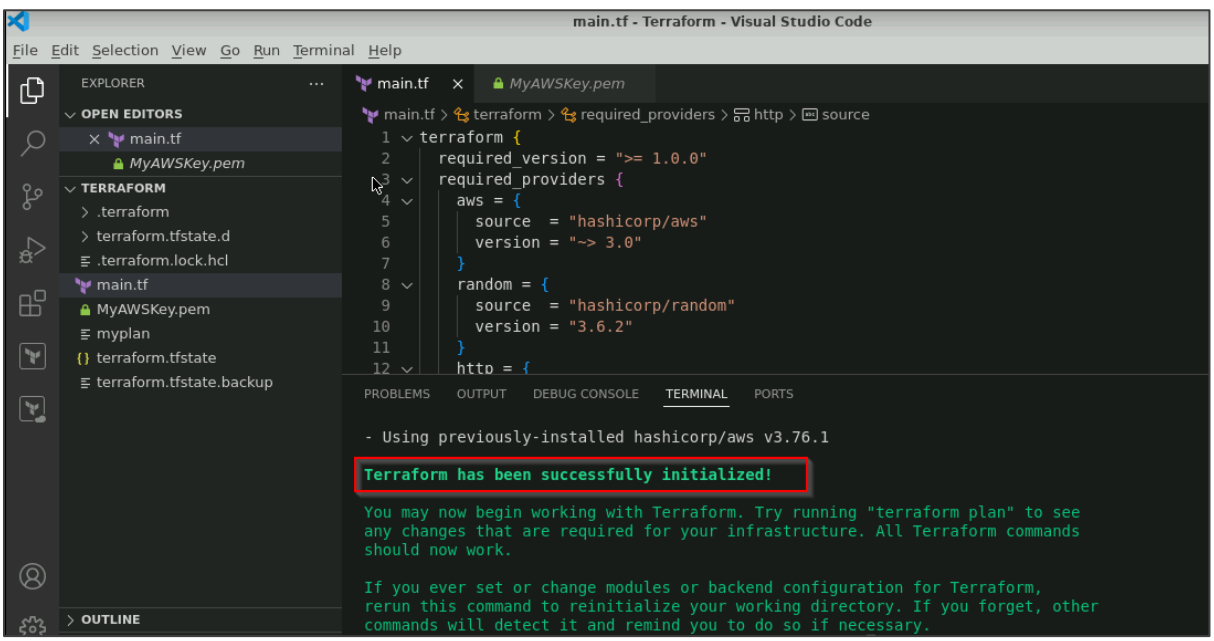


1.3 Run the **init** command to initialize the Terraform file:

terraform init

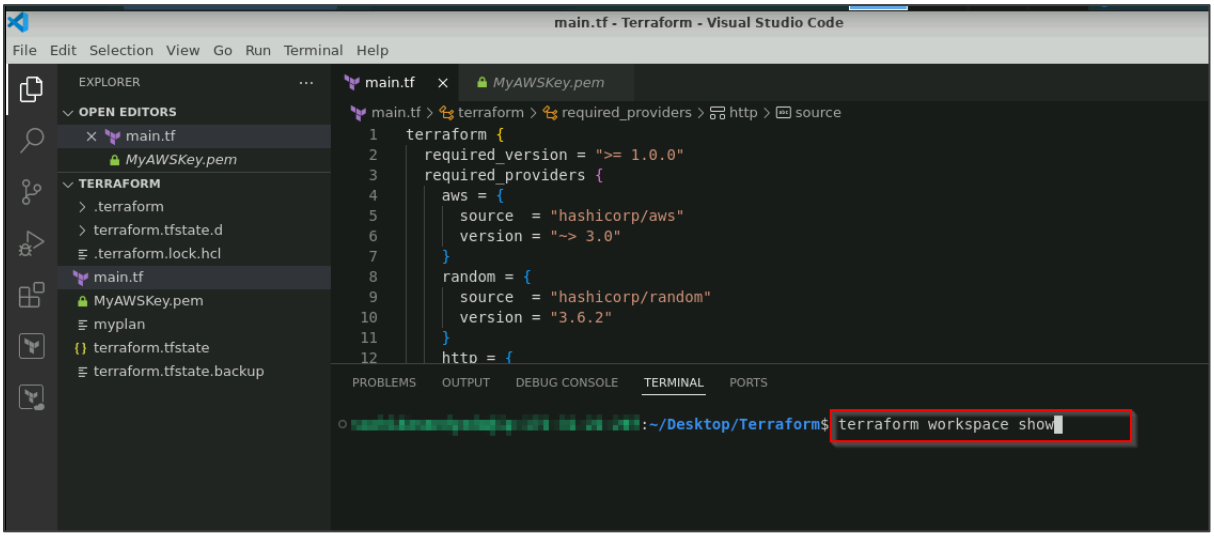


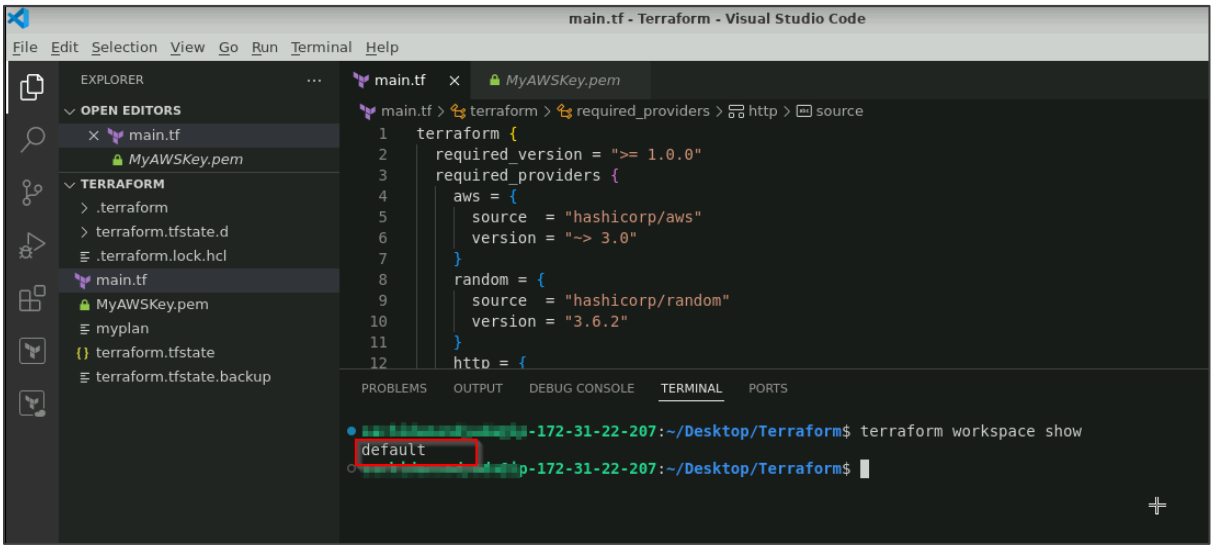
The terraform file is successfully initialized.



Step 2: Check the Terraform workspaces

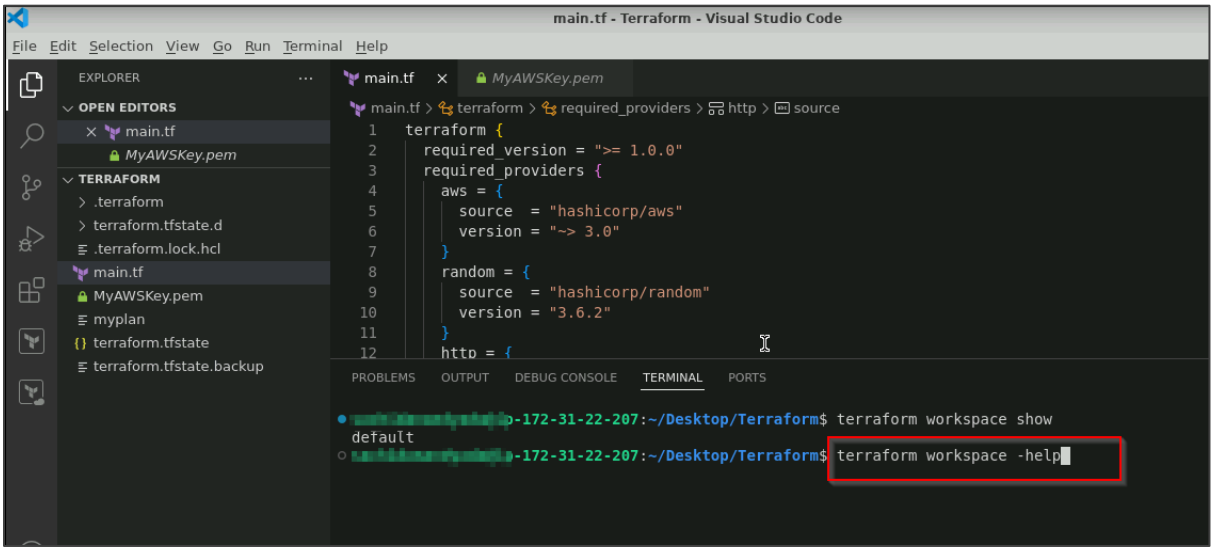
2.1 Open the terminal and run the command given below to check the current workspace:
terraform workspace show



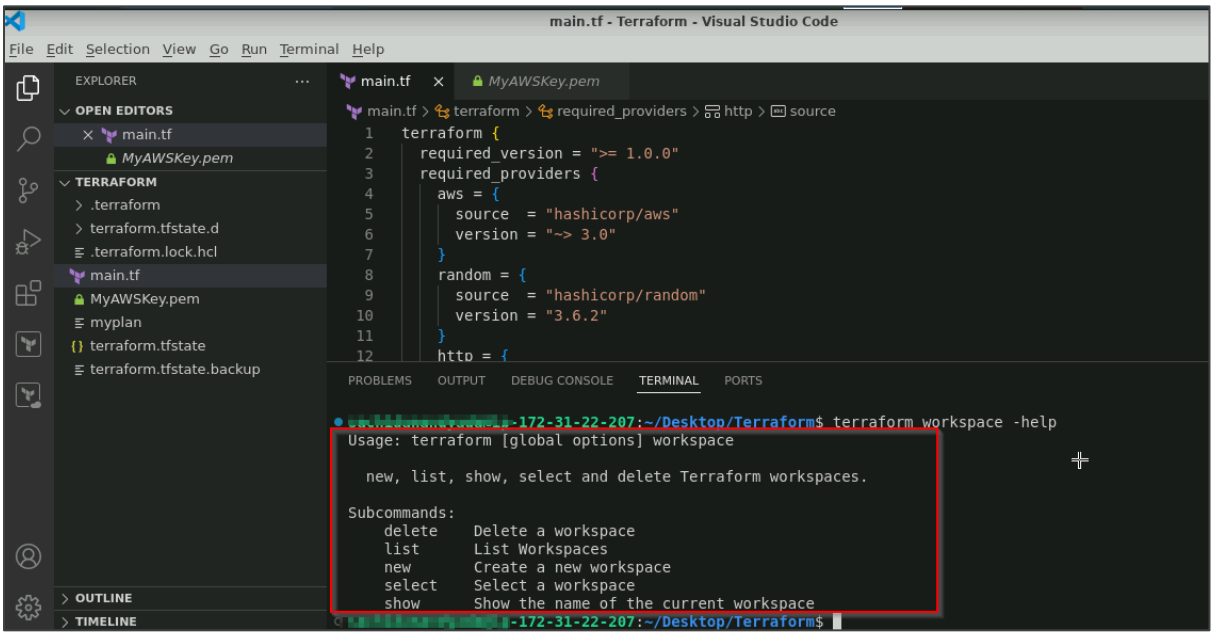


2.2 Run the command given below to display the subcommands available with the **terraform workspace** command:

terraform workspace -help



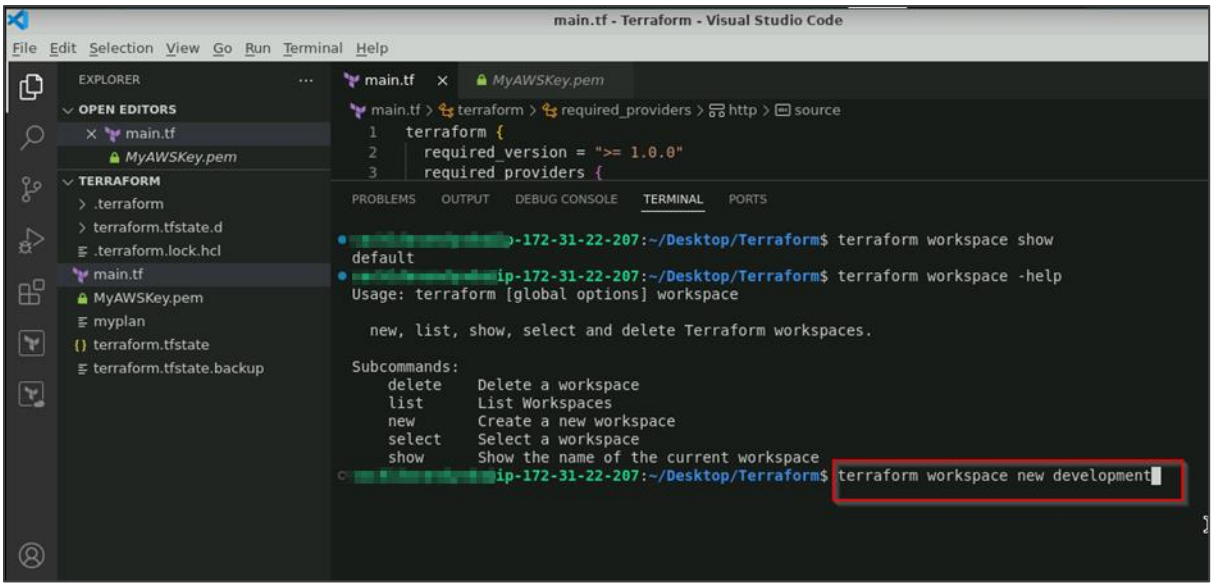
The **help** command displays the available subcommands.

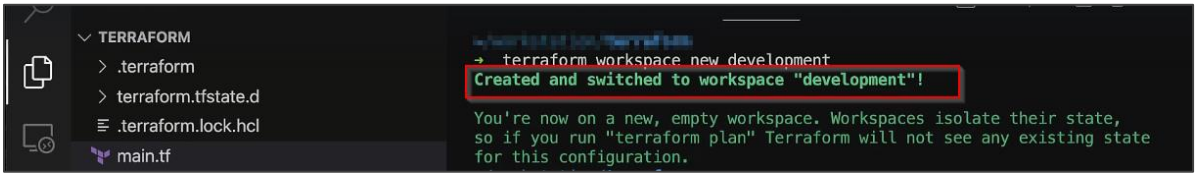


Step 3: Generate a new Terraform workspace for the development state

3.1 Open the terminal and run the command given below to generate a new Terraform workspace

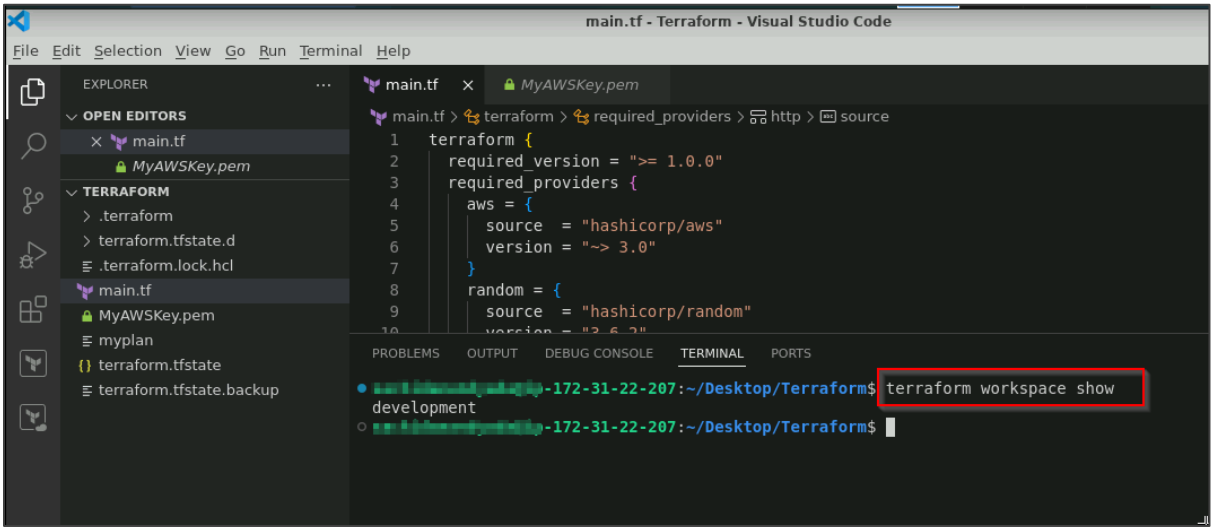
terraform workspace new development





3.2 Now run the command given below to see the current workspace:

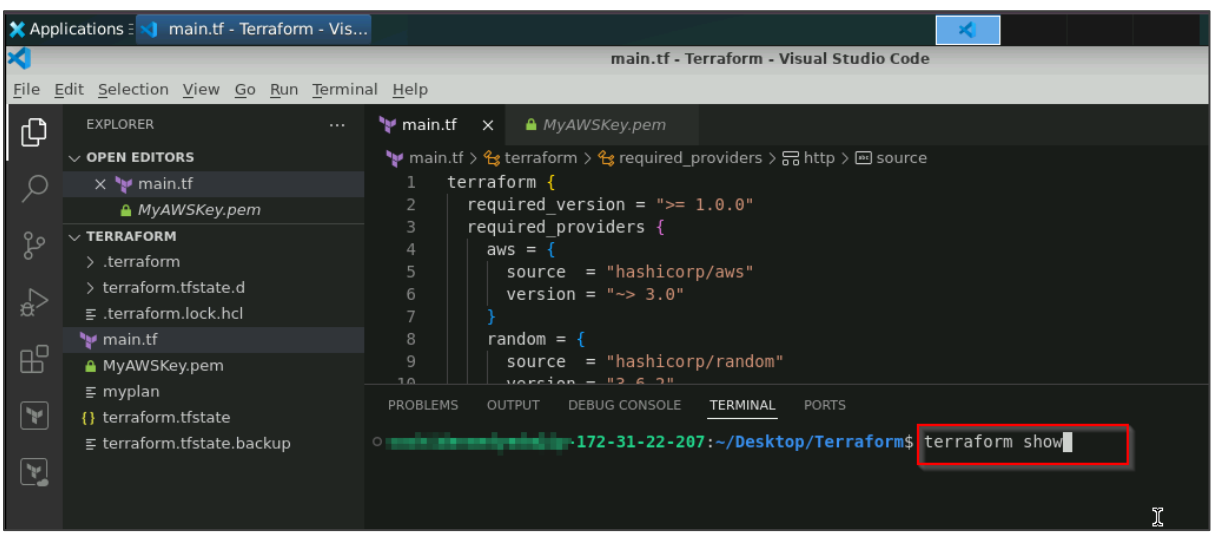
terraform workspace show

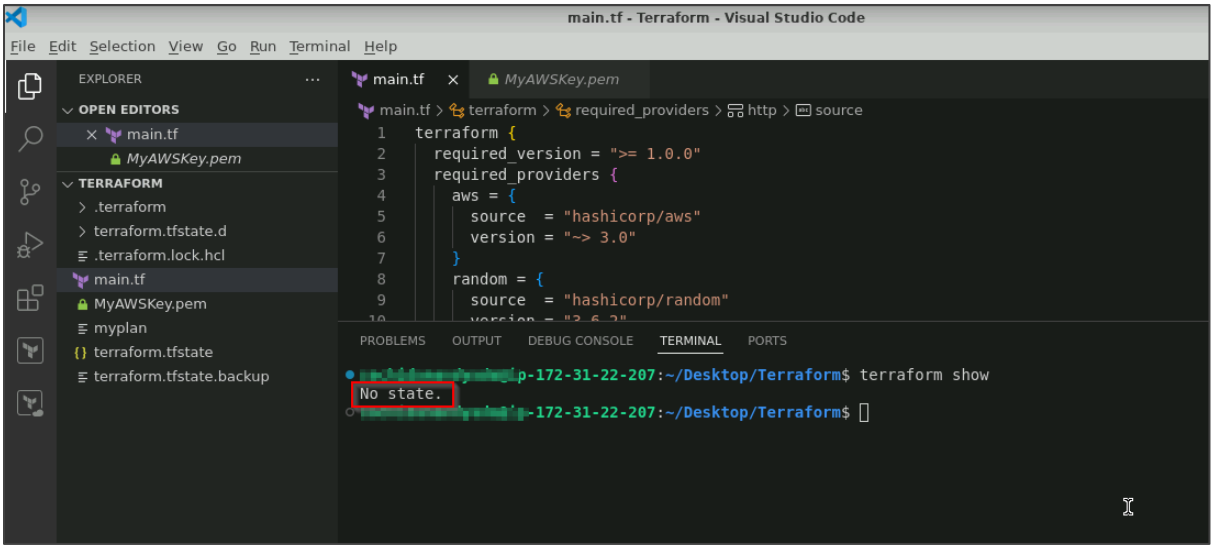


The newly generated workspace is available.

3.3 Now to check the state of the Terraform file run the command given below:

terraform show

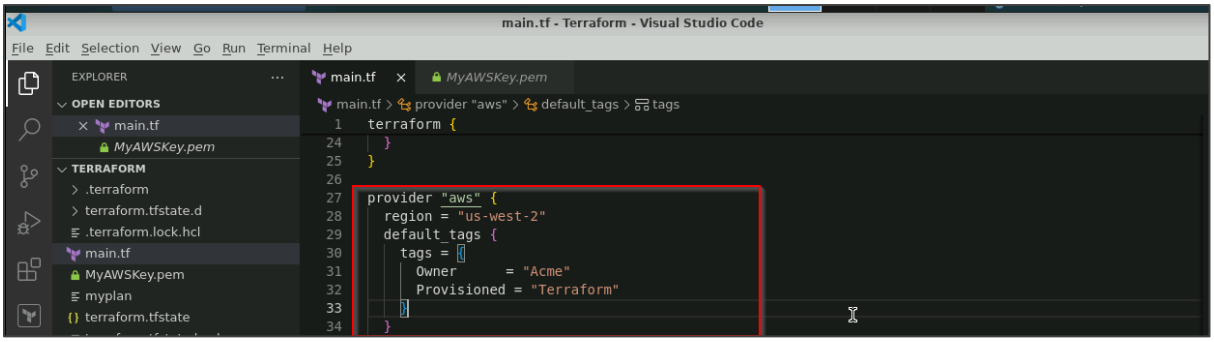




Step 4: Modify main.tf file

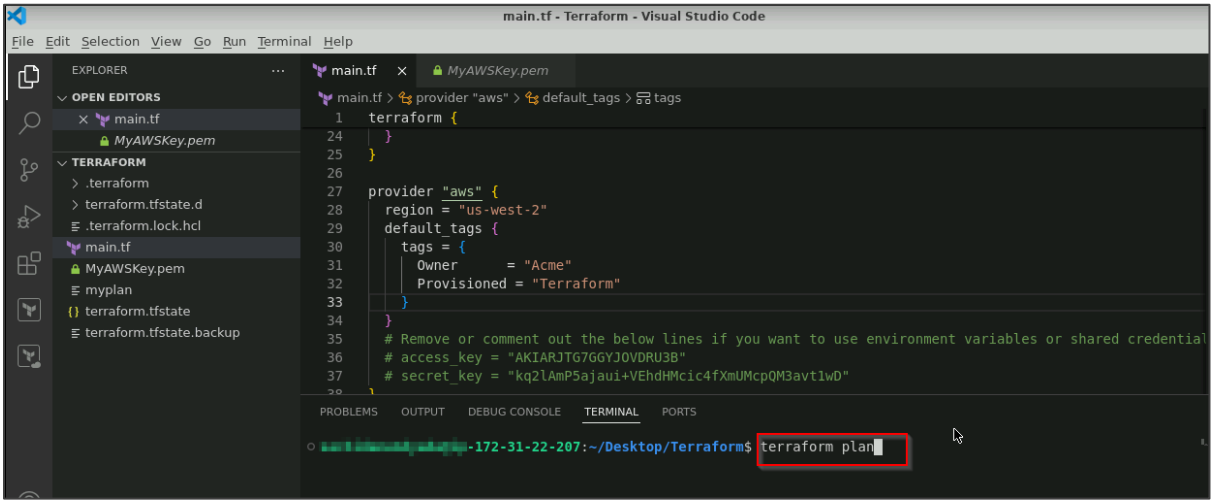
4.1 Open the **main.tf** file to make the changes in the **provider block** as per the code given below and save the file:

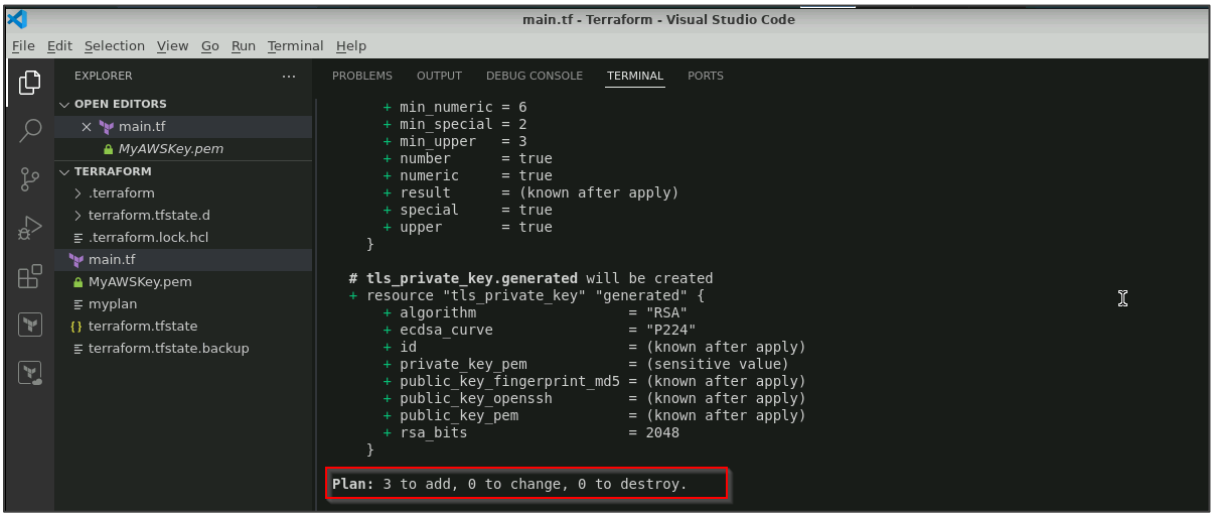
```
provider "aws" {
  region = "us-west-2"
  default_tags {
    tags = {
      Owner       = "Acme"
      Provisioned = "Terraform"
    }
  }
}
```



4.2 Run the **Terraform plan** command to see Terraforms run for execution as given below:

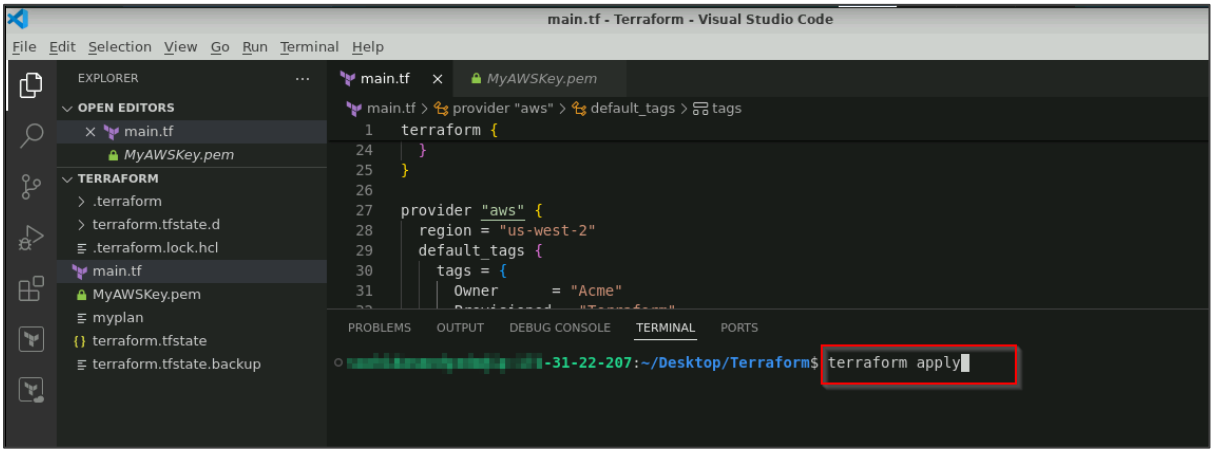
terraform plan



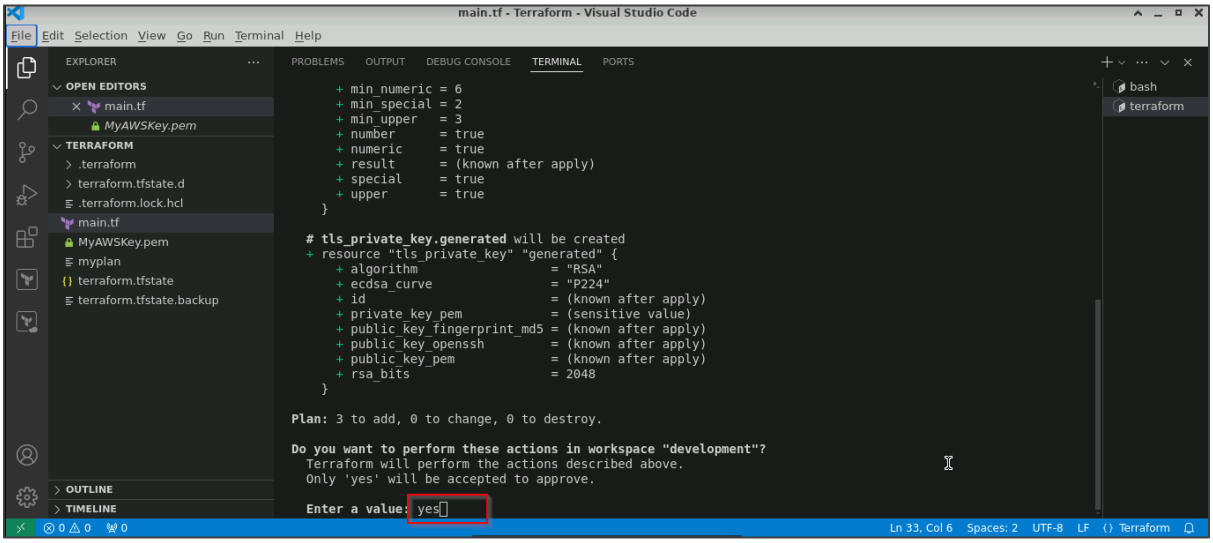


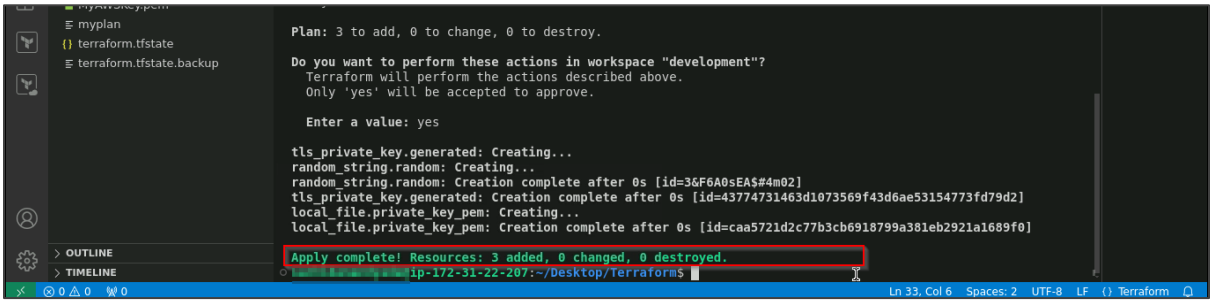
The **terraform plan** command runs successfully.

4.3 Open the terminal and run the command given below to apply the modifications:



Confirm as **yes** to proceed further.



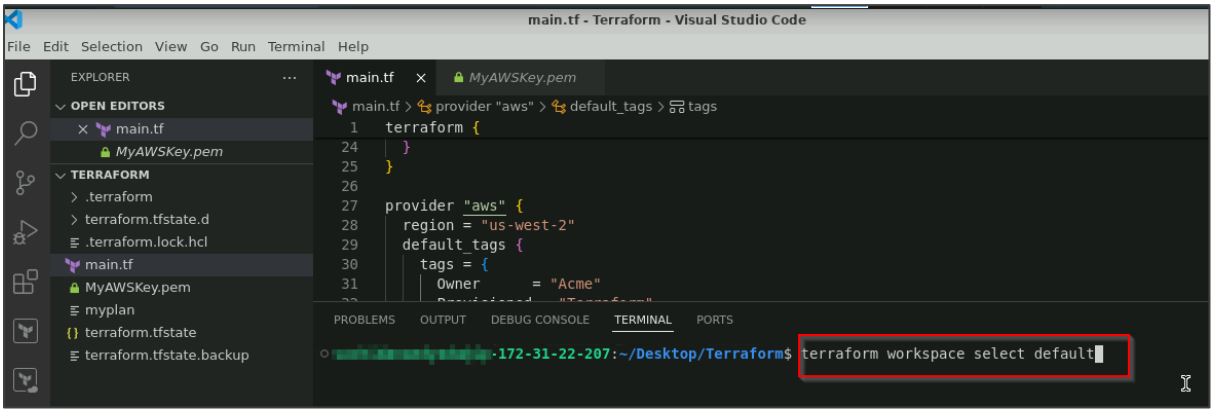


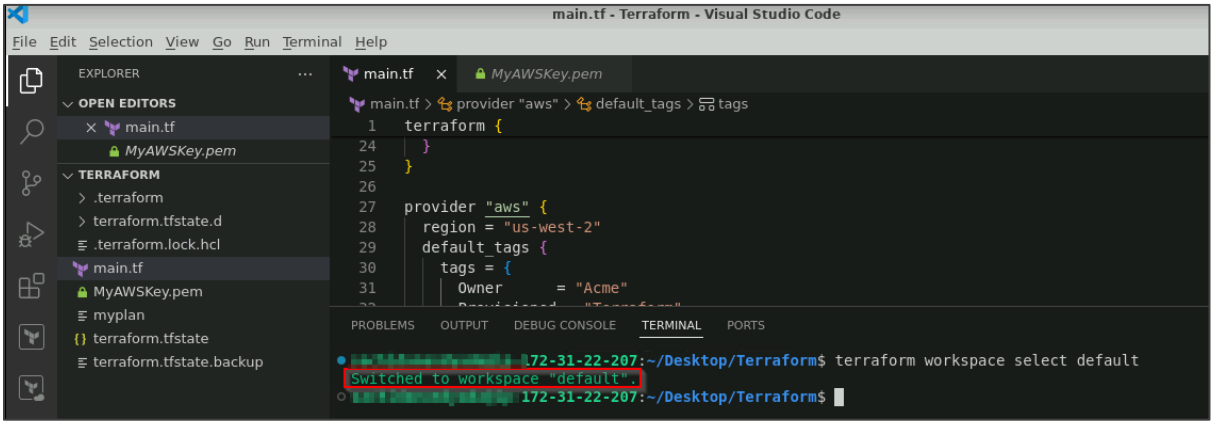
The apply command is completed successfully.

Step 5: Navigate between workspaces

5.1 Run the following command to navigate back to the default workspace:

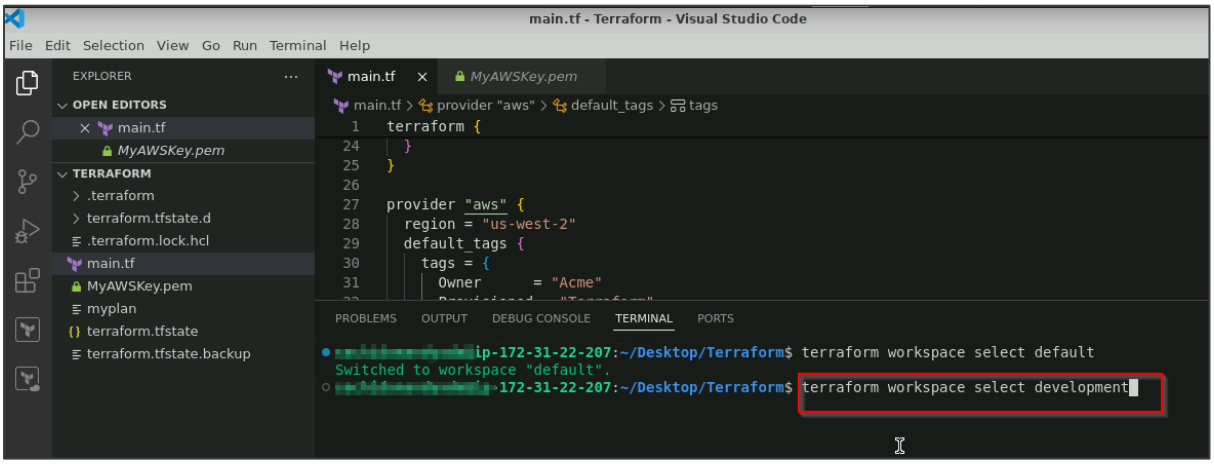
terraform workspace select default

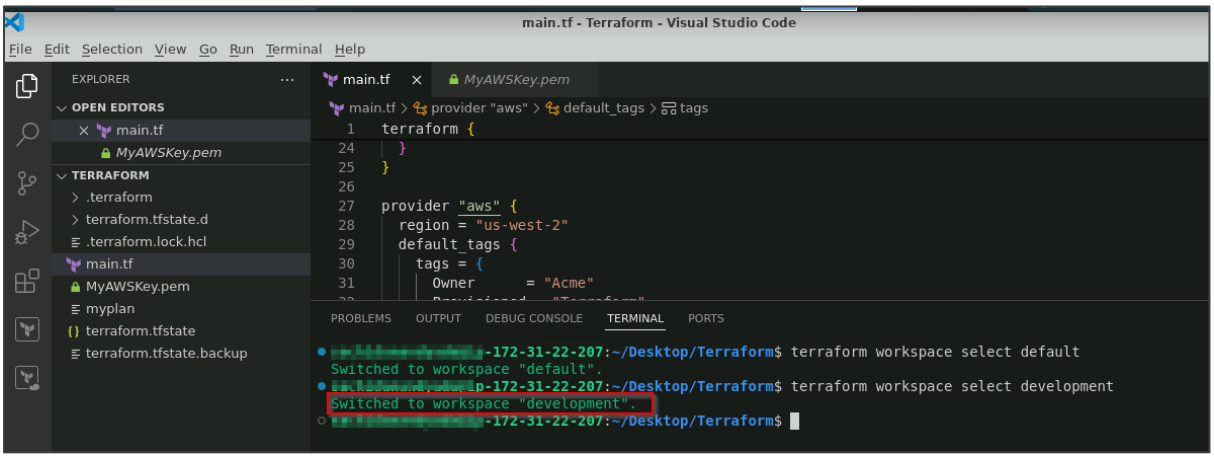




5.2 Run the following command to navigate back to the development workspace:

terraform workspace select development





By following the above steps, you have successfully utilized Terraform workspaces for environment separation, created and managed a development workspace, deployed infrastructure within it, and seamlessly switched between different workspaces.